

Introduction to Arduino in Robotics

Learning Objectives:

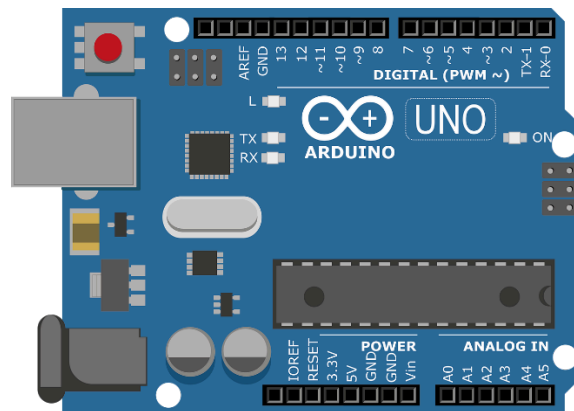
By the end of this module, students will be able to:

1. Understand the basic components and architecture of Arduino.
2. Install and configure the Arduino IDE.
3. Write and upload basic Arduino code.
4. Control an LED with simple commands.
5. Learn how to interact with input devices (such as buttons) to control output devices (such as LEDs).

1. Introduction to Arduino

What is Arduino?

Arduino is an open-source platform used for building electronic projects. It consists of both a physical programmable circuit board (microcontroller) and software (IDE) used to write and upload code to the board. Arduino boards are designed to be easy to use, making them a popular choice for robotics, automation, and DIY electronics projects.



Components of Arduino:

- **Microcontroller:** The heart of the board that runs the code.
- **Digital Pins (0 to 13):** Used to send or receive digital signals (high or low).

- **Analog Pins (A0 to A5):** Used to read analog signals (like sensors).
- **Power Supply Pins:** To power components.
- **USB Interface:** For connecting the board to a computer to upload code.

2. Setting Up Arduino IDE

What is the Arduino IDE?

The Arduino Integrated Development Environment (IDE) is the software used to write and upload programs to the Arduino board. The IDE supports various programming languages (mostly C/C++) and provides an interface for communication between the Arduino board and your computer.

Steps to Set Up Arduino IDE:

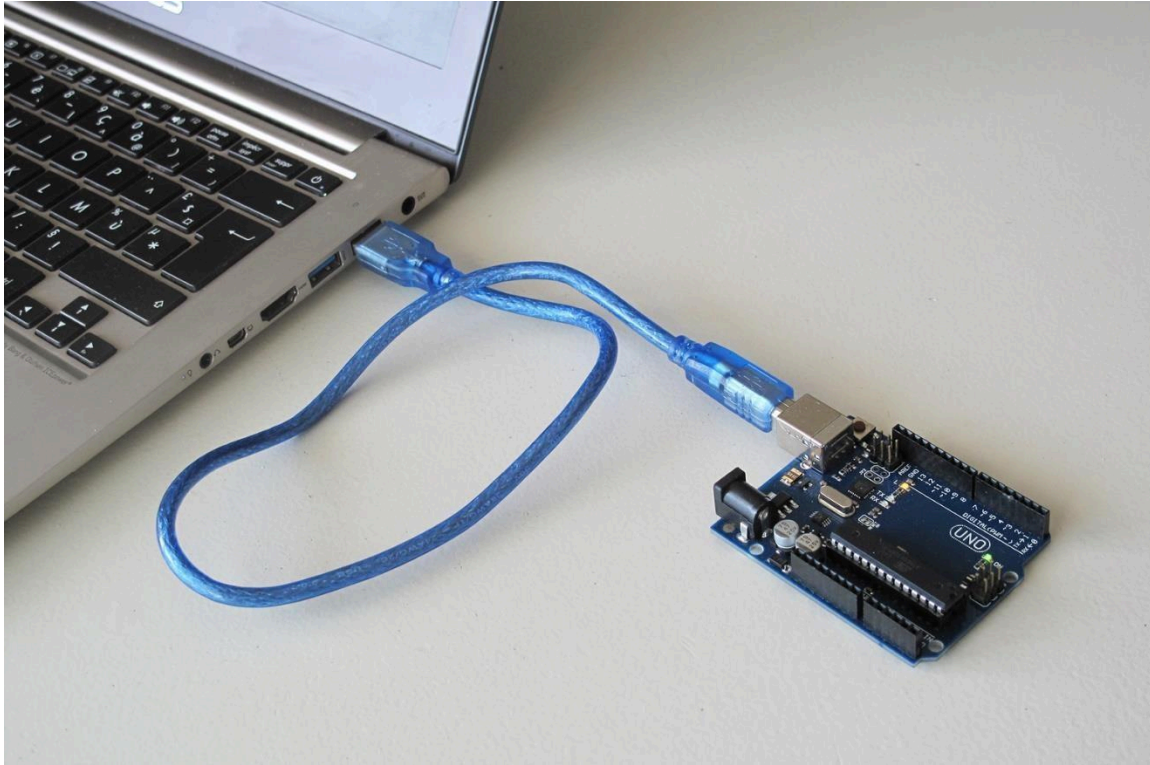
1. Download and Install Arduino IDE:

- Go to Arduino's official website.
- Download the appropriate version for your operating system (Windows, Mac, Linux).
- Follow the installation instructions.



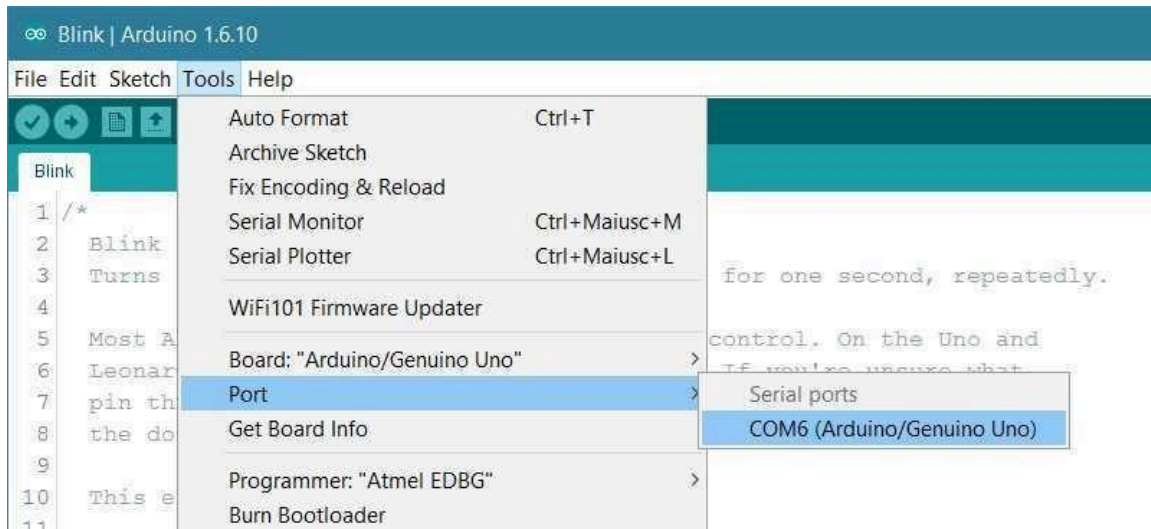
2. Connect the Arduino to Your Computer:

- Use a USB cable to connect your Arduino board to your computer.
- Make sure the board is recognized by your system.



3. Configure the IDE:

- Open Arduino IDE.
- Select your Arduino board: Go to **Tools > Board** and select your board (e.g., Arduino Uno).
- Select the correct port: Go to **Tools > Port** and choose the port that corresponds to your Arduino.

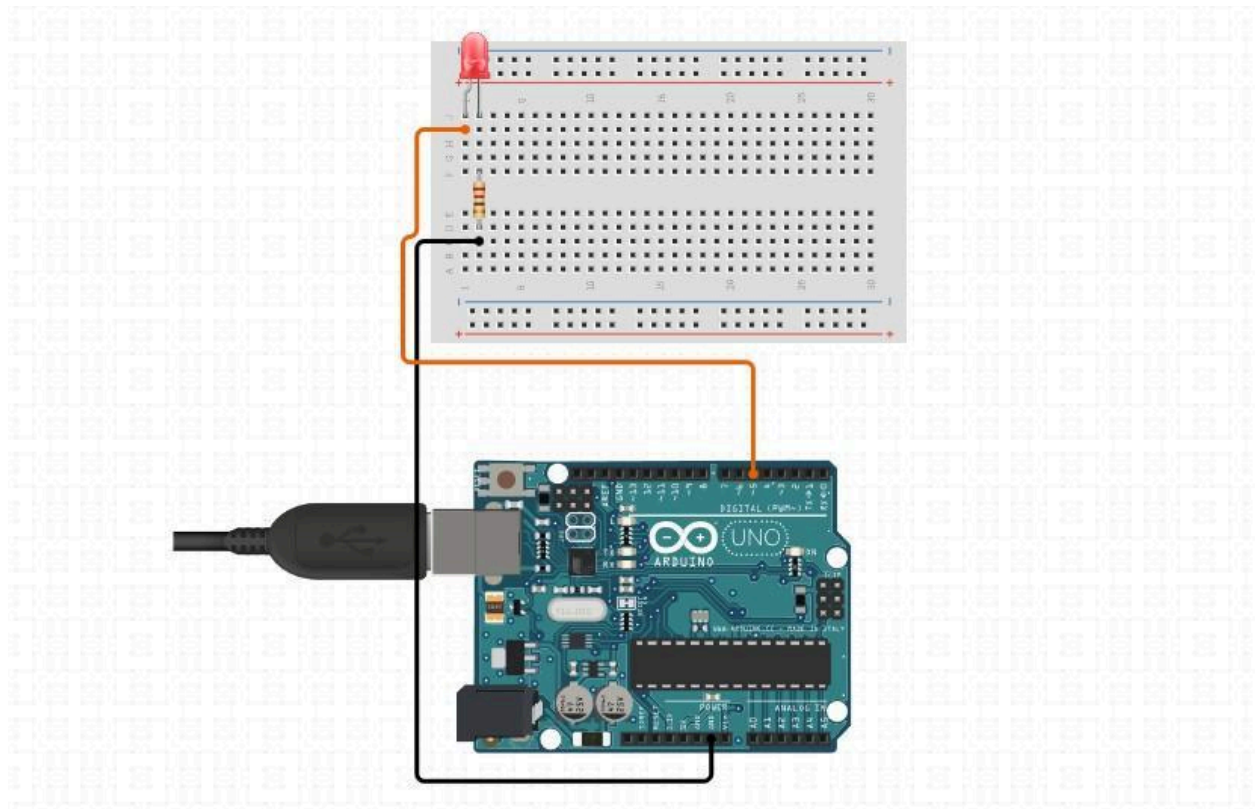


3. LED Control: Basic Blink Program

Components Required:

- Arduino board (e.g., Arduino Uno)
 - LED
 - 220Ω Resistor
 - Breadboard and jumper wires
- Wiring:**

1. Connect the long leg (anode) of the LED to pin 6 of the Arduino.
2. Connect the short leg (cathode) of the LED to a 220Ω resistor.
3. Connect the other end of the resistor to the GND pin on the Arduino

**Code:**

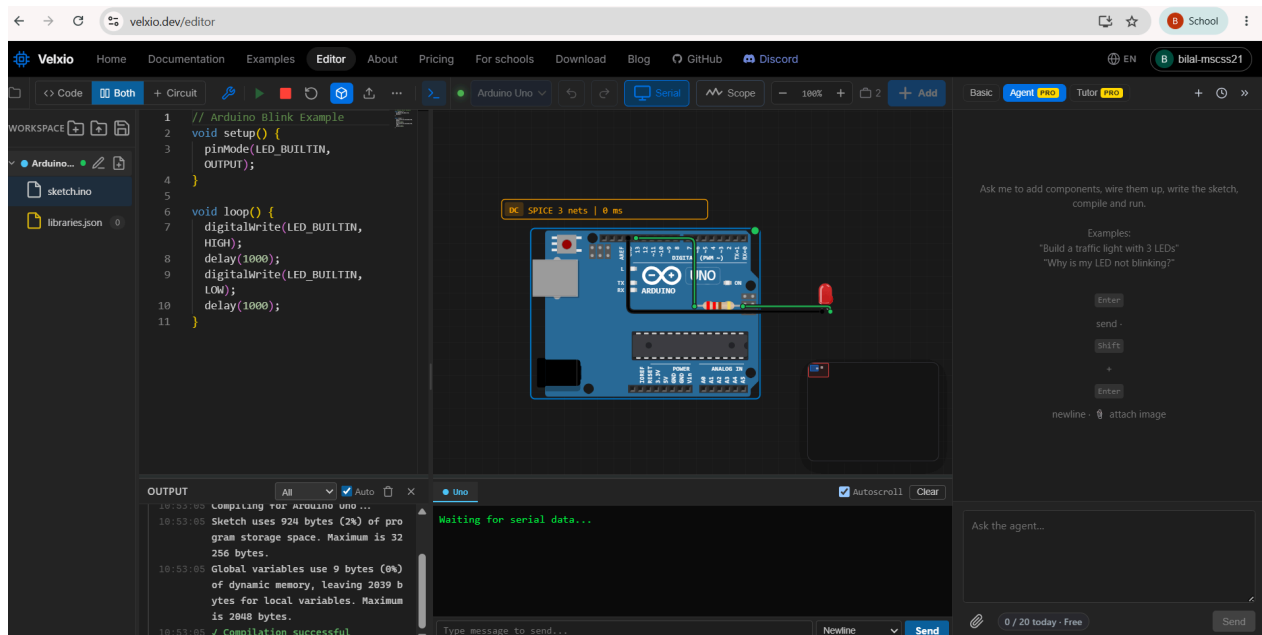
```
// Arduino Blink Example
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Explanation:

- `pinMode(ledPin, OUTPUT);` sets pin 6 as an output pin.
- `digitalWrite(ledPin, HIGH);` turns the LED on.
- `delay(1000);` pauses the program for 1000 milliseconds (1 second).
- The `loop()` function repeats the process indefinitely.

Output in Velxio:



Serial Communication of Arduino with PC

We can serially communicate with the Arduino and vice versa with the help of **Serial function** provided by the Arduino. Serial library contains various functions, but basic functions are as follows.

- **Serial.begin(buad_rate):** Used to define buad_rate, a speed is set to synchronize the communication between Arduino and PC, normally it set to 9600.
- **Serial.print(message):** Used to send a message on serial port.
- **Serial.println(message):** Used to send a message on serial port on new line.

Example Code

```
// C++ code
//
void setup()
```

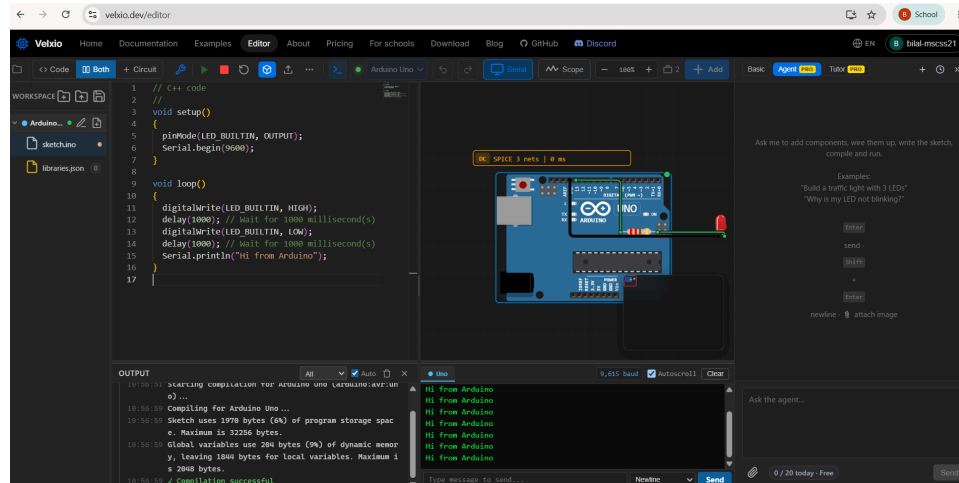
```

{
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
}

void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
  Serial.println("Hi from Arduino");
}

```

Output in Velxio



Serial Plotter

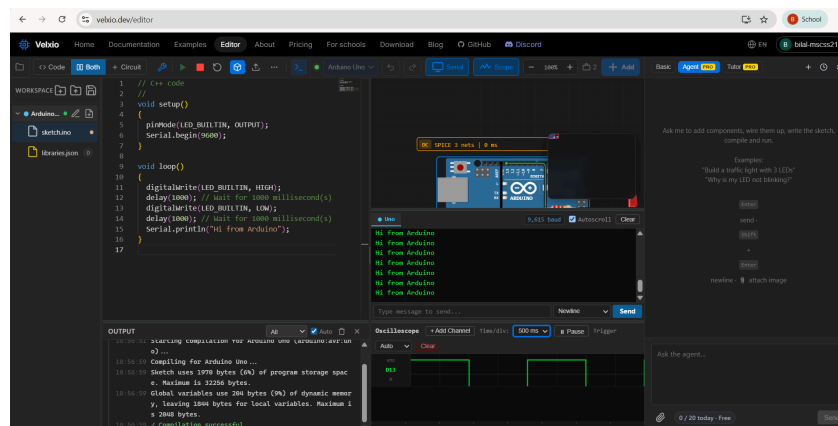
Use the Arduino Serial Plotter to visualize numerical data generated by the Arduino in real time. Send sensor readings or other variable values through the Serial interface and observe their changes as a graph. This will help you understand how sensor values and system outputs vary over time without requiring advanced electronics knowledge.

Velxio Serial Plotter

Use the Velxio Serial Plotter in the same way to visualize serial data within the simulation environment. Run your simulated Arduino program, generate sensor or variable readings, and observe the resulting graph in real time. This provides an easy way to analyze system behavior and understand sensor data before working with physical electronic instruments such as oscilloscopes.

We can use them to plot following

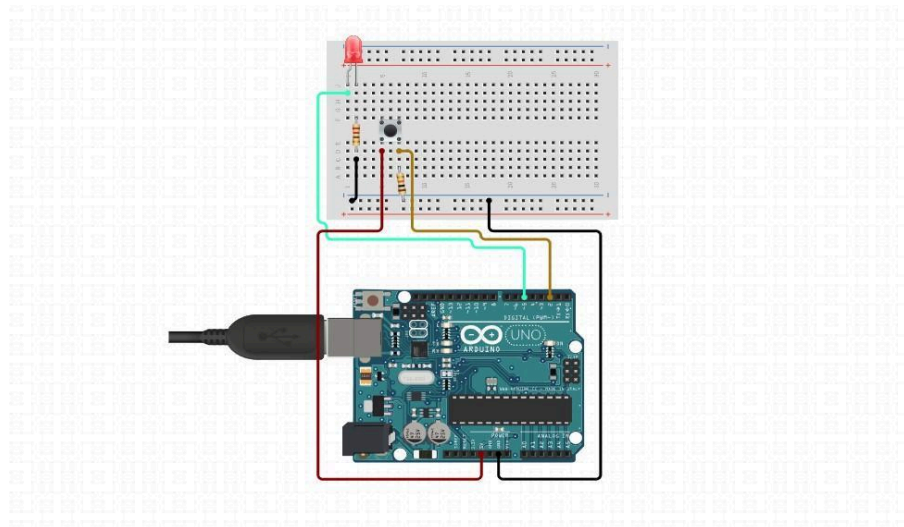
- **Monitor sensor readings** such as temperature, light, distance, and potentiometer values.
- **Observe changes over time** and identify trends or variations in sensor data.
- **Compare multiple sensor signals** simultaneously.
- **Debug programs** by visualizing variables and checking whether they behave as expected.
- **Analyze analog signals** and observe changes in ADC readings.
- **Visualize PWM or control-related values** generated by a program.
- **Study system responses** when an input or sensor condition changes.



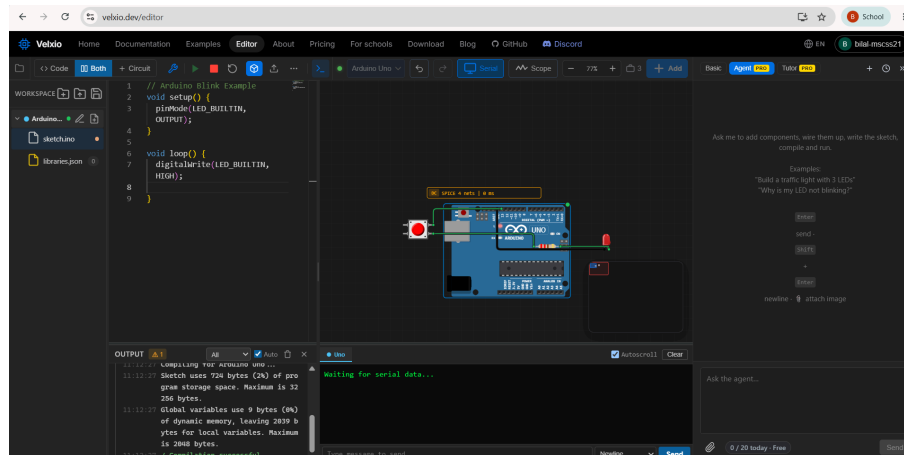
4. LED Control Using a Button

Components Required:

- Arduino board
 - LED
 - 220 Ω Resistor
 - Pushbutton
 - 10k Ω Resistor (for pull-down configuration)
 - Breadboard and jumper wires
- Wiring:**
1. Connect the anode of the LED to pin 6 and the cathode to the 220 Ω (Optional) resistor. The other side of the resistor goes to GND.
 2. Connect one side of the button to pin 2 and the other side to GND.
 3. Add a 10k Ω pull-down resistor (Optional) between pin 2 and GND to ensure a stable LOW signal when the button is not pressed.



Output in Velxio:



Class Assignment Code:

```
// Set the button pin as an input
pinMode(buttonPin, INPUT);
}

void loop() {
  // Read the button state
  buttonState = digitalRead(buttonPin);
  Serial.println(buttonState);
  // If the button is pressed, turn the LED on
```

```
if (buttonState == HIGH) {  
  digitalWrite(ledPin, HIGH);  
} else {  
  digitalWrite(ledPin, LOW);  
} }
```

Task 1: Run it, understand it and explain it.

Task 2: Create Traffic light signal project using multiple LEDs

Task 3: Create a simple home appliances board using multiple buttons and multiple LEDs.

Conclusion:

Through this module, you have learned how to:

- Set up and configure the Arduino IDE.
- Write basic Arduino programs to control an LED.
- Implement PWM to control the brightness of an LED.
- Use a button to control the state of an LED.

This knowledge forms the foundation for more advanced robotics projects where sensors and actuators interact to create automated systems.

Further Reading and Resources:

- Arduino Official Documentation: <https://www.arduino.cc>
- Arduino IDE: <https://www.arduino.cc/en/Guide>
- Language and function details with usage: <https://docs.arduino.cc/language-reference/>